

Print at 100% / Actual size, SINGLE-sided. Crease along the grey line, fold so both printed faces are outward, glue, then cut. This bar must measure exactly 80 mm.

[illegible]

Every AI algorithm needs a way to represent the state of the game. In this case, we can use a 2D array to represent the board. The board is a 3x3 grid of squares, each containing a character representing a piece or being empty.

The first step is to initialize the board. We'll use a function called `init_board` to set up the initial state. This function will create a 3x3 array where each element is either 'X' (player), 'O' (computer), or '.' (empty).

Next, we need a way to check if a player has won. We'll use a function called `check_win`. This function will iterate through the rows, columns, and diagonals of the board to see if all three positions contain the same non-empty character.

We also need a way to choose the best move for the computer. We'll use a function called `minimax`. This function uses a recursive approach to explore all possible moves and counter-moves, evaluating the final state of the board to determine the optimal path.

Finally, we'll put it all together in a main loop that alternates between the player's turn and the computer's turn until the game ends.

```
def init_board():  
    board = [['.' for _ in range(3)] for _ in range(3)]  
    return board  
  
def check_win(board, player):  
    # Check rows  
    for row in board:  
        if row == [player] * 3:  
            return True  
    # Check columns  
    for col in range(3):  
        column = [board[row][col] for row in range(3)]  
        if column == [player] * 3:  
            return True  
    # Check diagonals  
    diagonal1 = [board[i][i] for i in range(3)]  
    diagonal2 = [board[i][2-i] for i in range(3)]  
    if diagonal1 == [player] * 3 or diagonal2 == [player] * 3:  
        return True  
    return False  
  
def minimax(board, depth, maximizing):  
    if check_win(board, 'X'):  
        return -10  
    elif check_win(board, 'O'):  
        return 10  
    elif not any('.' in row for row in board):  
        return 0  
  
    if maximizing:  
        max_eval = float('-inf')  
        for i in range(3):  
            for j in range(3):  
                if board[i][j] == '.':  
                    board[i][j] = 'O'  
                    eval = minimax(board, depth + 1, False)  
                    board[i][j] = '.'  
                    max_eval = max(max_eval, eval)  
        return max_eval  
    else:  
        min_eval = float('inf')  
        for i in range(3):  
            for j in range(3):  
                if board[i][j] == '.':  
                    board[i][j] = 'X'  
                    eval = minimax(board, depth + 1, True)  
                    board[i][j] = '.'  
                    min_eval = min(min_eval, eval)  
        return min_eval  
  
def get_best_move(board):  
    best_val = float('-inf')  
    best_move = None  
    for i in range(3):  
        for j in range(3):  
            if board[i][j] == '.':  
                board[i][j] = 'O'  
                val = minimax(board, 0, False)  
                board[i][j] = '.'  
                if val > best_val:  
                    best_val = val  
                    best_move = (i, j)  
    return best_move  
  
def play_game():  
    board = init_board()  
    while not check_win(board, 'X') and not check_win(board, 'O') and not all('.' in row for row in board):  
        # Player's turn  
        print("Player's turn")  
        show_board(board)  
        move = input("Enter row and column indices separated by space: ")  
        row, col = map(int, move.split())  
        if board[row][col] != '.':  
            print("Invalid move! Cell already occupied.")  
            continue  
        board[row][col] = 'X'  
  
        # Computer's turn  
        print("Computer's turn")  
        row, col = get_best_move(board)  
        board[row][col] = 'O'  
  
        show_board(board)  
  
    show_board(board)  
    if check_win(board, 'X'):  
        print("Player wins!")  
    elif check_win(board, 'O'):  
        print("Computer wins!")  
    else:  
        print("It's a draw!")
```

[illegible][illegible]